Exploring Tourist Destinations in Tirunelveli Through Graph Theory

Name: Roshin Farheen M, P. Sangeethaa

Reg No: 2024176031, 2025166049

ABSTRACT

This project is about creating a tourism network using a web application. In this system, tourist places are shown as nodes and the connections between them are shown as edges using graph theory.

The data is taken from an Excel file. We use Python, Flask, and Network to build the project. The system calculates important values like Degree Centrality, Betweenness Centrality, and Closeness Centrality to find how important each place

The user can:

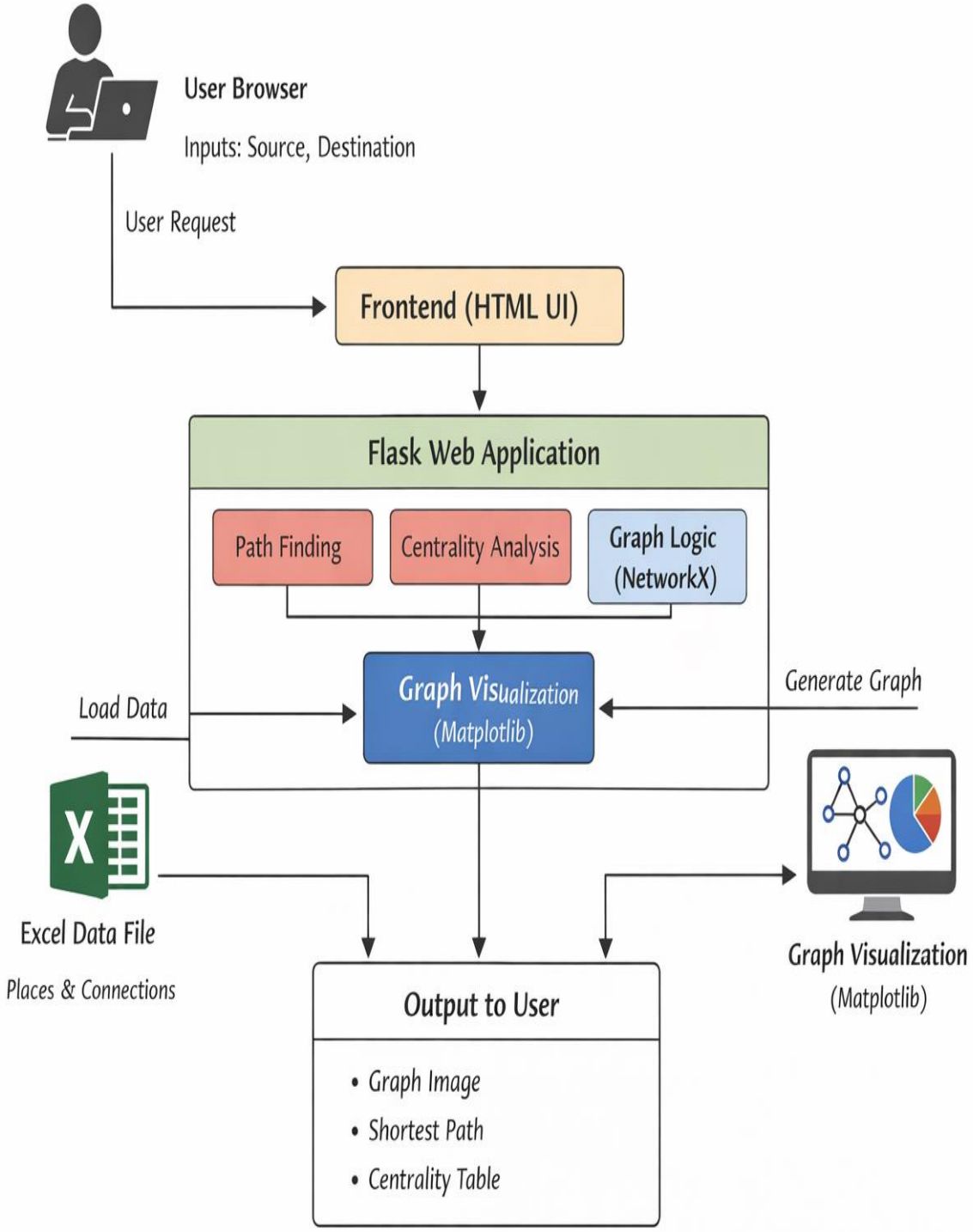
- Find the shortest path between two places
- See paths from one place to all other places
- View the full tourism network as a graph

This project helps users to understand routes between places and makes travel planning easier.

OBJECTIVES

- To represent tourist places as nodes in a graph
- To connect places using edges
- To find shortest path between two places
- To find paths from one place to all other places
- To calculate centrality values
- To display the graph visually
- To build a simple web application using Flask

ARCHITECTURE



ARCHITECTURE EXPLANATION

The architecture of this project shows how the system works from user input to output.

First, the user opens the website in the browser and gives input like source and destination places. This input is sent as a request to the system.

Next, the request goes to the Frontend (HTML page). This is the part where the user interacts with the system. It takes input and displays the results.

After that, the request is handled by the Flask Web Application (Backend). This is the main part of the system where all processing happens.

Inside the Flask application, there are three main parts:

- Path Finding → finds shortest path between places
- Centrality Analysis → calculates importance of each place
- Graph Logic (Network) → creates and manages the graph

Then, the system loads data from the Excel file, which contains:

List of places

- Connections between places
- Using this data, the graph is created and processed.
- After processing, the graph is sent to Graph Visualization (Matplotlib). This part draws the graph and saves it as an image.
- Finally, the output is shown to the user.

The user can see:

- ✓ Graph image
- ✓ Shortest path
- ✓ Centrality values

User → Frontend → Flask Backend → Process (Graph + Path + Centrality) → Visualization → Output

METHODOLOGY

First, we collect the tourism data and store it in an Excel file. The file has two sheets:

- Places (list of locations)
- Connections (links between places)

Next, we create a graph using Network:

Add all places as nodes

Add all connections as edges

Then, we calculate centrality values:

- ✓ Degree Centrality
- ✓ Betweenness Centrality
- ✓ Closeness Centrality

After that, we implement path finding:

- Shortest path between two places
- Paths from one place to all places
- We also draw the graph using Matplotlib and save it as an image.

Finally, we create a Flask web app:

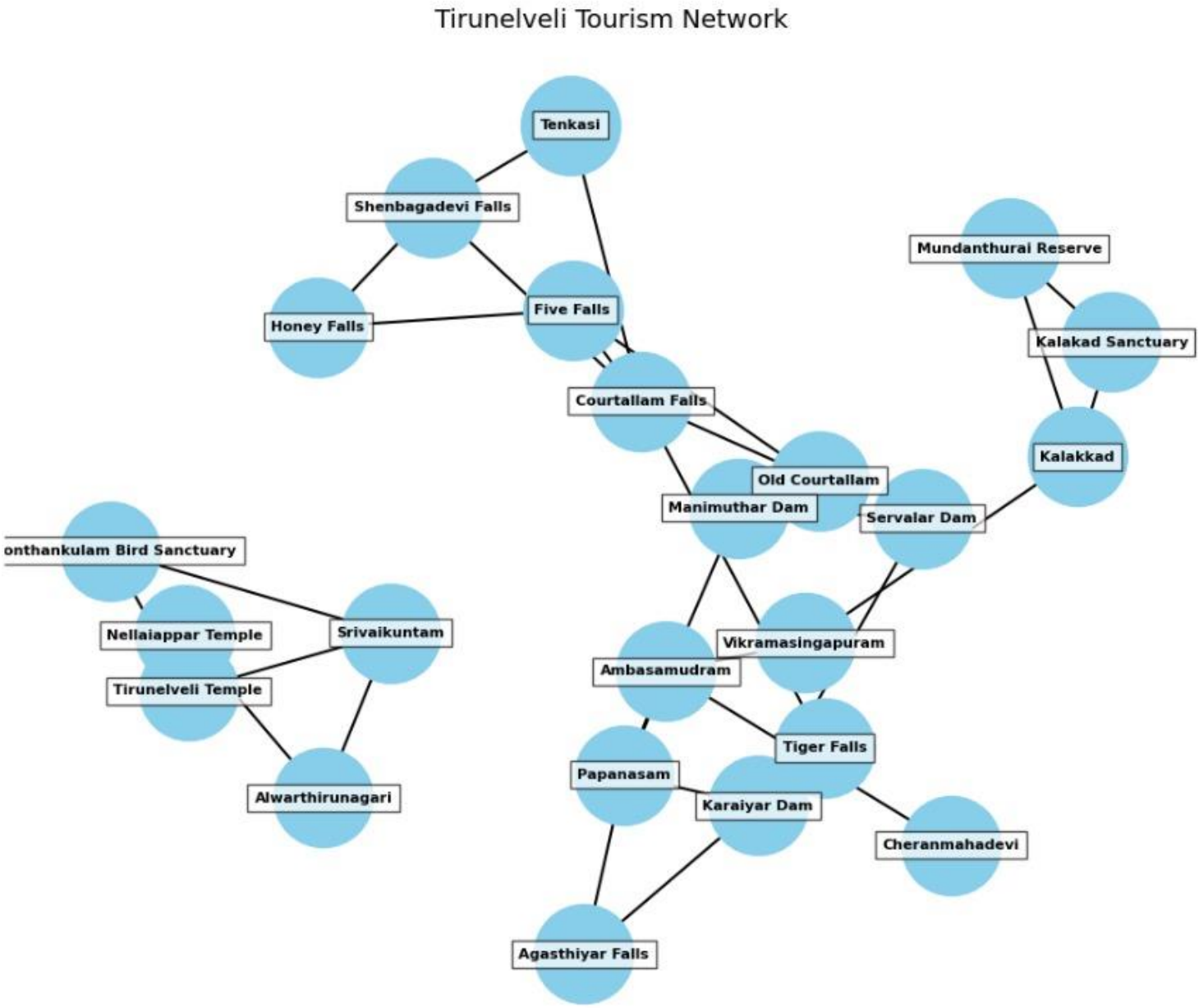
- User gives input through forms
- Results are shown on the webpage

PSEUDOCODE

```
START
Read Excel file
Get list of places
Get list of connections
Create empty graph G

Add all places as nodes to G

Add all connections as edges to G


Function calculate_metrics:
    Find degree centrality
    Find betweenness centrality
    Find closeness centrality
    Store and return result


Function find_path(source, destination):
    Try to find shortest path
    If found → return path
    Else → return "No path"


Function find_all_paths(source):
    For each place in graph:
        Find shortest path
        Store result
    Return all paths


Function draw_graph:
    Draw nodes and edges
```

```
    Add labels
    Save image

When user opens website:
    Show graph
    Show centrality table

If user enters source and destination:
    Show shortest path

If user enters one source:
    Show paths to all places

Run Flask app

END
```

📍 Graph


Tirunelveli Tourism Network
Tenkasi
Shenbagadevi Falls
Honey Falls
Five Falls
Courtallam Falls
Mundanthurai Reserve
Kalakad Sanctuary
Kalakkad
Old Courtallam
Manimuthar Dam
Servalar Dam
onthankulam Bird Sanctuary
Nellaiappar Temple
Srivaikuntam
Tirunelveli Temple
Alwarthirunagari
Vikramasingapuram
Ambasamudram
Tiger Falls
Papanasam
Karaiyar Dam
Cheranmahadevi
Agasthiyar Falls

🔍 Find Path (A → B)

Source | Destination | Find

🌍 Paths from One Place to ALL

Enter Source | Find All

📊 Centrality Analysis

Place	Degree	Betweenness	Closeness
Courtallam Falls	0.227	0.037	0.234
Papanasam	0.182	0.110	0.216
Tirunelveli Temple	0.136	0.006	0.145
Five Falls	0.136	0.009	0.182
Kalakkad	0.136	0.069	0.162
Srivaikuntam	0.136	0.006	0.145
Karaiyar Dam	0.136	0.019	0.168
Shenbagadevi Falls	0.136	0.009	0.182
Ambasamudram	0.136	0.126	0.227
Old Courtallam	0.091	0.000	0.164
Agasthiyar Falls	0.091	0.000	0.162
Manimuthar Dam	0.091	0.015	0.162
Alwarthirunagari	0.091	0.002	0.121
Kalakad Sanctuary	0.091	0.000	0.126
Koonthankulam Bird Sanctuary	0.091	0.000	0.121
Mundanthurai Reserve	0.091	0.000	0.126
Servalar Dam	0.091	0.002	0.134
Nellaiappar Temple	0.091	0.002	0.121
Vikramasingapuram	0.091	0.091	0.198
Honey Falls	0.091	0.002	0.149
Tenkasi	0.091	0.000	0.164
Cheranmahadevi	0.045	0.000	0.157
Tiger Falls	0.045	0.000	0.136